CPU Scheduling Simulator

An interactive web application for visualizing and comparing CPU scheduling algorithms.

Jonathan Koral

About Me & My Role

Solo developer - Computer Science, Brooklyn College

Who Am I:

- Jonathan Koral - CS student at BC
- Supervised by Prof. Minh Le
- Solo project - I take care of the frontend, backend, testing, and deployment
- My Contact: jon.koral1214@gmail.com
- Supervisor Contact: minh.le@brooklyn.cuny.edu

My Responsibilities:

- Design the system architecture
- Implement scheduling algorithms in java
- Build the style of the web frontend
- Write and maintain the REST API
- Write JUnit test cases for every algorithm
- Manage the project schedule week-by-week
- Prepare all documentation & design diagrams

Problem Statement

Why does this problem exist?

The Problem
Students studying CPU scheduling algorithms struggle to verify whether the Gantt charts they write are correct. The only resources available are static textbook diagrams, class notes, and online videos. None of which can check a student's specific input.

The Solution
An interactive web simulator that accepts any set of processes, runs the selected scheduling algorithm, and instantly returns a Gantt chart, process metrics, and summary statistics, giving the user personalized feedback for any input.

Students
Verify their Gantt charts and compare algorithms

Instructors
Present live demonstrations of scheduling behavior

CS Learners
Build up their understanding of preemptive vs non-preemptive scheduling

Computer Science Concepts This Project Applies

Algorithms & Complexity

FCFS: $O(n \log n)$

SJF: $O(n^2 \log n)$

SRTF: $O(n^2)$

Software Engineering

Design patterns (Strategy Pattern), REST API design, and separation of controller/service/model layers

Software Testing

JUnit test cases with 46 unit tests covering boundary values, edge cases, and correctness of algorithm output

Operating Systems

CPU scheduling, gantt charts, process states, waiting time, turnaround time, and throughput

Data Structures

ArrayDeque for ready queues, HashMap for algorithm lookup and remaining burst tracking in SRTF, ArrayList for Gantt entries

Web Technologies

Full-stack: Java Spring Boot backend communicates with a JS frontend by JSON over HTTP; I added CORS handling

Tools & Technologies

Backend

- Java 21
- Spring Boot 4.0.3
- Spring Web (REST)
- Lombok
- Maven

Frontend

- HTML
- CSS
- JavaScript
- Fetch API (HTTP calls)

Testing

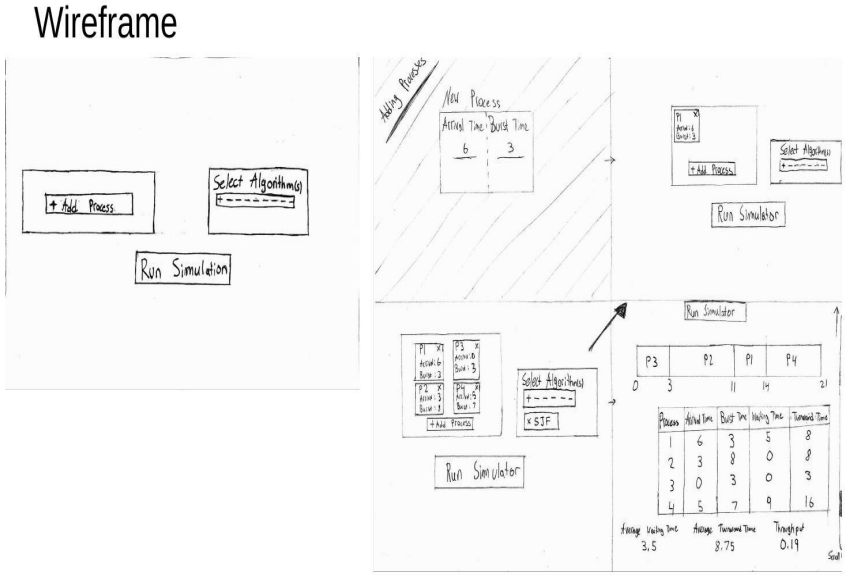
- JUnit 5
- Postman (API testing)
- Web Browser testing

Dev Tools

- IntelliJ IDEA
- GitHub

Wireframe

Initial Wireframe Sketch



What Changed from the Wireframe

- Positioning of “Select Algorithm” and “Add Process” changed. “Select Algorithm is now on top of “Add Process”. “Run Algorithm” is now next to ‘Select Algorithm”
- The design of the modal has changed from a horizontal input format to a vertical one. I also added the input option of priority which is optional for all algorithms besides Priority scheduling
- Gantt charts have Idle blocks to show when the CPU is idling when no processes are available at that time frame.

Current Version

CPU Scheduling Simulator

Select Algorithm: FCFS Run Algorithm

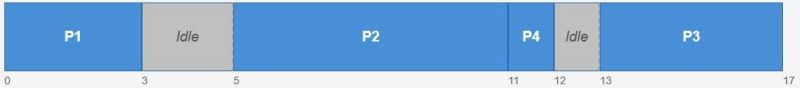
Processes

P1	Arrival: 0	Burst: 3	Priority: 0	Delete
P2	Arrival: 5	Burst: 6	Priority: 0	Delete
P3	Arrival: 13	Burst: 4	Priority: 0	Delete
P4	Arrival: 9	Burst: 1	Priority: 0	Delete

+ Add Process

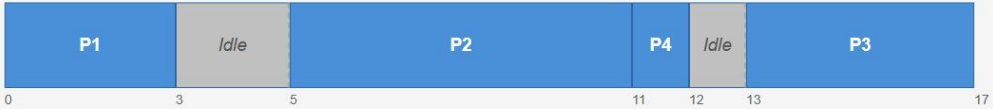
Results

Gantt Chart



Results

Gantt Chart



Process Table

Process	Arrival Time	Burst Time	Priority	Waiting Time	Turnaround Time
P1	0	3	0	0	3
P2	5	6	0	0	6
P4	9	1	0	2	3
P3	13	4	0	0	4

Avg Waiting Time
0.50

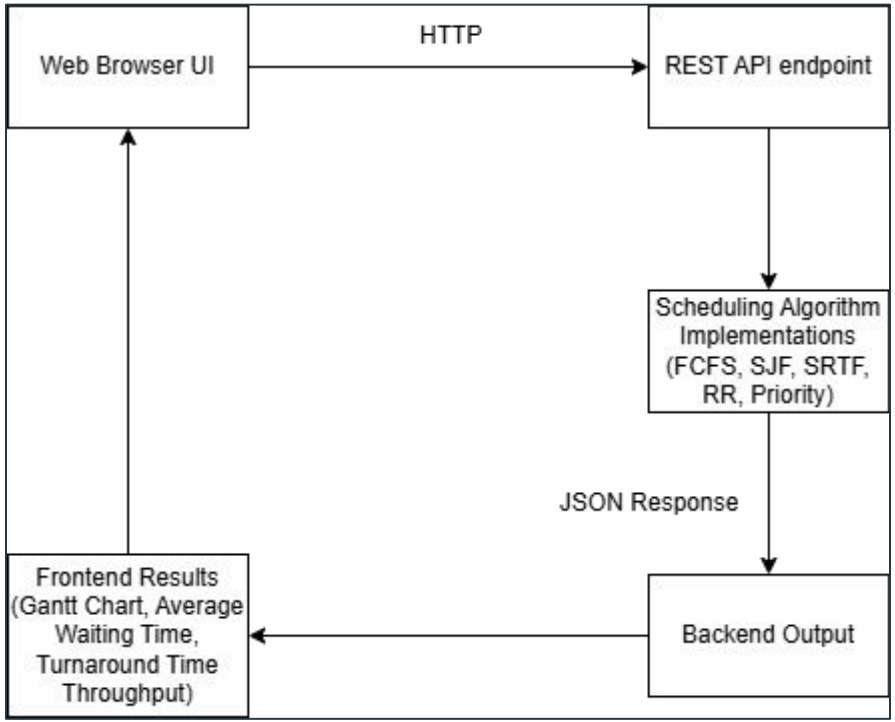
Avg Turnaround Time
4.00

Throughput
0.2353

All output is generated live by the Spring Boot backend. Nothing is hardcoded on the frontend.

System Architecture

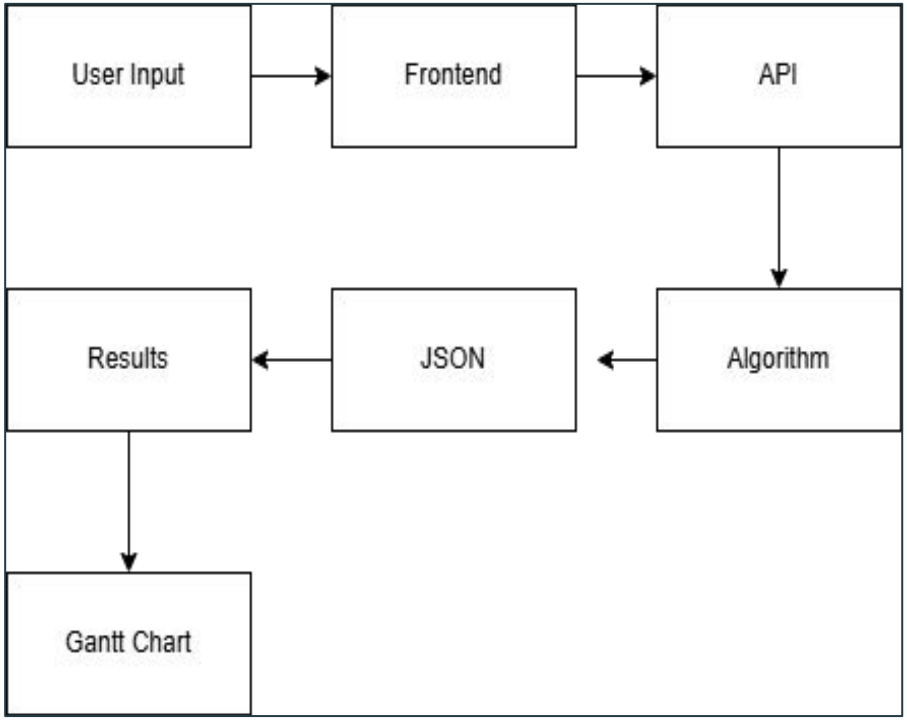
How the frontend, backend, and algorithms connect



Key Components

- Web Browser UI
  - HTML/CSS/JS - sends JSON POST to backend
- Simulation Controller
  - REST endpoint: POST /api/simulate
- SchedulerService
  - HashMap lookup to select the correct algorithm
- AlgorithmStrategy
  - Interface implemented by FCFS, SJF, SRTF, RR, Priority
- ScheduleResult
  - DTO returned as JSON with Gantt, table, and stats

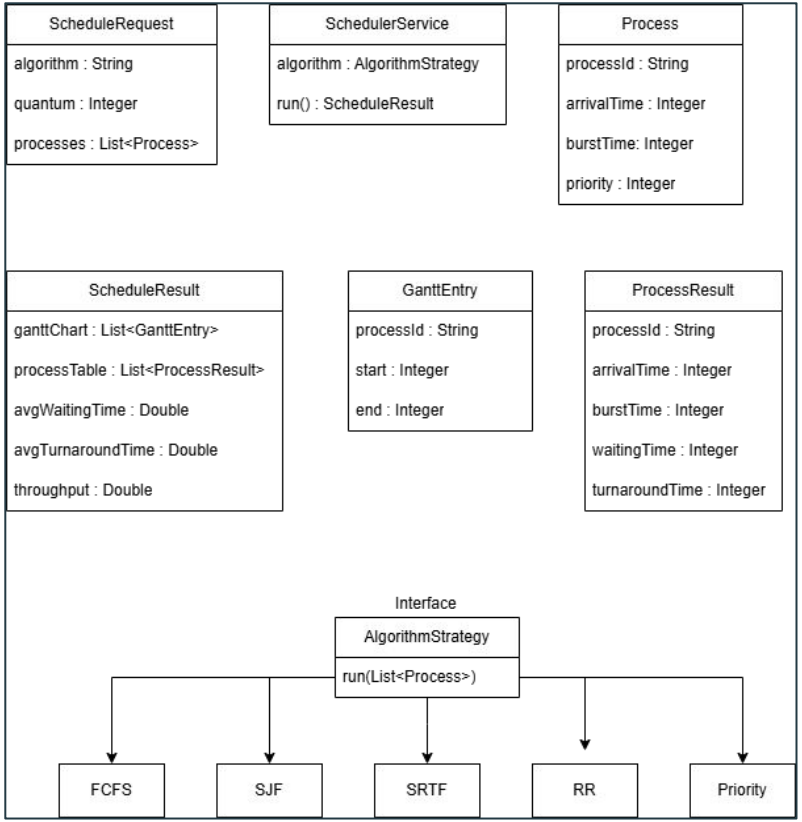
Data Flow Diagram



Data at Each Stage

- User Input
  - processId, arrivalTime, burstTime, priority, algorithm
- Frontend
  - Validates fields, builds JSON request body
- API
  - Sends ScheduleRequest from in JSON to backend
- Algorithm
  - Takes a list of processes and returns a ScheduleResult
- JSON Response
  - ganttChart, processTable, averages, throughput
- Results UI
  - Renders Gantt chart, table, and stat cards

Class Diagrams - Backend Structure



Design Patterns Used

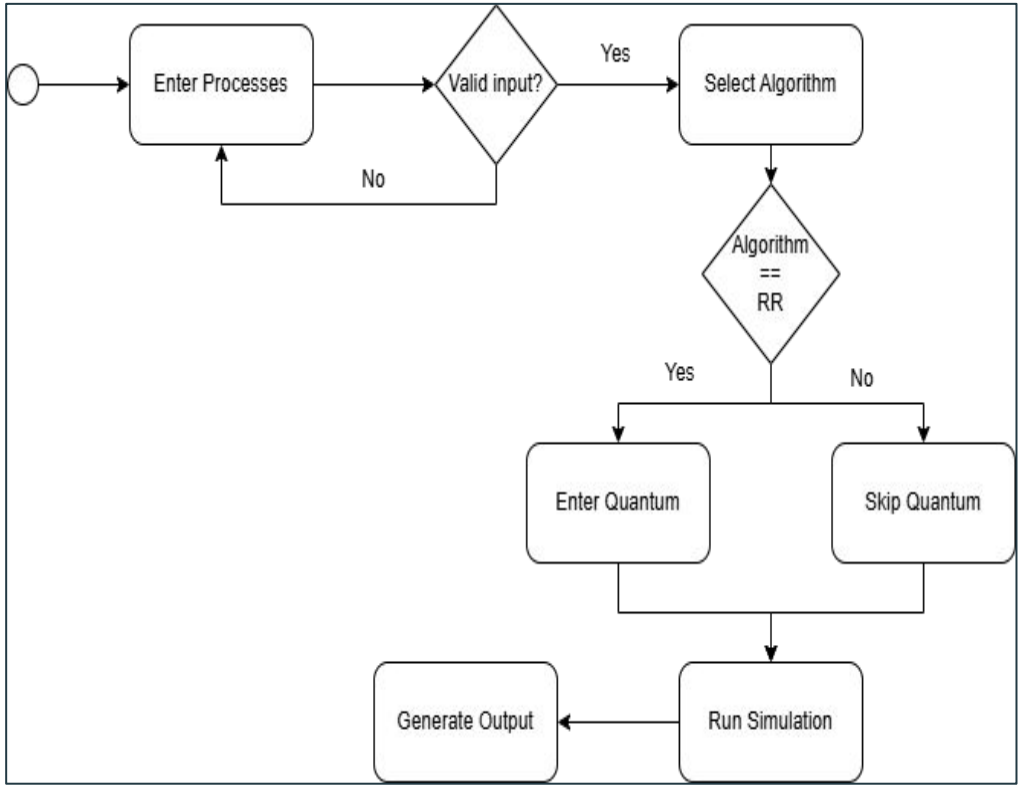
Strategy Pattern

Each algorithm implements a shared AlgorithmStrategy interface. SchedulerService holds a HashMap<String, AlgorithmStrategy> and selects the right one at runtime. Adding a new algorithm (e.g. Priority) only requires a new class, no other code changes.

Key Classes:

- AlgorithmStrategy (interface)
- FCFCAlgorithm, SJFAlgorithm, SRTFAlgorithm
- SchedulerService
- SimulationController
- Process, GanttEntry, ProcessResult
- ScheduleRequest, ScheudleResult

Activity Diagram



Activity Highlights

- User enters processes and selects algorithm
- If Round Robin is selected, then quantum input appears
- Frontend validates all fields before sending to backend
- Backend selects the correct algorithm by use of HashMap
- Algorithm runs and packages results into ScheduleResult
- Frontend renders Gantt chart, table, and stat cards

Scheduling Algorithms Design & Efficiency

FCFS

Runtime: O(n log n)

Non-preemptive

The most expensive step is the initial sort by arrival time. Sorting n processes takes O(n log n) — that's just the cost of any efficient sort like merge sort or TimSort which Java uses under the hood. After that, the algorithm loops through the list once which is just O(n). Since O(n log n) dominates O(n), the overall runtime is O(n log n).

SJF

Runtime: O(n^2 log n)

Non-preemptive

This one is more expensive because the sorting happens repeatedly. SJF is non-preemptive, so every time the CPU becomes free, it needs to look at all the processes that have arrived so far and pick the one with the shortest burst time. To do that, it sorts the ready queue. In the worst case, this happens n times (once per process finishing), and each sort costs O(n log n). So you get n sorts × O(n log n) per sort = O(n^2 log n) total.

SRTF

Runtime: O(n^2)

Preemptive

SRTF works at every single tick of time, it scans the entire ready queue to find whichever process has the least remaining burst time. That scan costs O(n). In the worst case, the total number of time units the CPU runs for is proportional to n (the sum of all burst times scaled to the number of processes). So you get O(n) ticks × O(n) scan per tick = O(n^2).

Toughest Bug (SRTF Gantt Chart Splitting)

The Bug

SRTF is preemptive so a process can be interrupted mid-run and resume later. My initial Gantt chart implementation was merging the interrupted and resumed segments into one block, making the chart wrong.

Example: P1 runs 0–3, P2 preempts 3–5, P1 resumes 5–9.

My code was outputting: [P1: 0–9] instead of [P1: 0–3], [P2: 3–5], [P1: 5–9].

The Fix

```
// Close current Gantt entry on preemption

if(currentProcessId == "" || !currentProcessId.equals(shortestProcess.getProcessId())) {

        if(currentProcessId != "") {

       // Add Entry when switching

       // not just when finished

            GanttEntry ganttEntry = new GanttEntry(currentProcessId, curr, end);

            ganttChart.add(ganttEntry);

        }

        currentProcessId = shortestProcess.getProcessId();

        curr = end;

    }
```

A Gantt entry must be created wherever the running process running changes

Testing in JUnit

Edge Cases Tested

- Single process (arrives at t=0 and late)
- All processes arrive at same time
- All arrive at same non-zero time
- Processes with max burst time (99) and max arrival (99)
- Process arrives exactly when previous finishes

Correctness Tests

- Gantt chart order and start/end times
- verify that each individual process got the correct waiting and turnaround times
- Average waiting time
- Average turnaround time
- Throughput = n / last end time

Algorithm-Specific

- SRTF: preemption occurs when a new process arrives with strictly less remaining time
- SRTF: tie-break by process ID (smaller wins)
- SRTF: Gantt splits on preemption
- SJF: if equal burst, then earlier arrival runs first

Tentative Schedule

Weeks 5–8

Tentative Schedule

Week 5	Set up the Backend • Create the Spring Boot project • Set up the folder structure • Create the classes • Create the REST endpoints	4-5 hours
Week 6	Implement FCFS • Test the algorithm with a set of inputs Create the Demo Recording video	2-6 hours 5 minutes
Week 7	Implement SJF • Test the algorithm with a set of inputs	2-5 hours
Week 8	Set up the Frontend • Create the user input forms • Add dropdown menu for selecting algorithms • Test logic and validate user inputs Peer Evaluation	4-8 hours 1 ~ hour(s)

Weeks 9–12

Tentative Schedule

Week 9	Implement SRTF • Test the algorithm with a set of inputs	2-5 hours
Week 10	Work on Frontend • Fixing up the style for the Gantt chart • Make the interface more user friendly • Update the design (layout/colors)	4-8 hours
Week 11	Finish up the full draft of the slides • Fix up slides and add whatever is required	15 ~ hours (total work after 1st draft)
Week 12	Implement Round Robin • Update the frontend to accept quantum time when Round Robin is the selected algorithm. • Test the algorithm with a set of inputs	2-6 hours

Weeks 13–15

Tentative Schedule

Week 13	Create the second Demo Recording video Implement Priority Scheduling • Update frontend to accept priority as input for processes • Test the algorithm with a set of inputs	5 minutes 2-5 hours
Week 14	Final work on Frontend • Update the user interface • Make any improvements on the layout • Implement pop up error messages	4-7 hours
Week 15	Final Slides • Strengthen the slides that need work • Add new material as required Create the Final Demo Recording video Submit the project to be graded	5-7 hours 5 minutes 1 minute

Completed:

- FCFS
- SJF
- SRTF
- Working Frontend
- JUnit Tests

Planned:

- RR
- Priority
- RR & Priority JUnit Tests
- Second Demo Recording
- Final UI Polish

Use Case 1 : Student Checks Homework

A student uses my CPU scheduling simulator to verify their homework solutions.

Student enters the following processes to the simulator:

- P1: arrival 0, burst 5
- P2: arrival 2, burst 3
- P3: arrival 4, burst 1

The student selects a scheduling algorithm and runs the simulation.

The output the student receives:

- Gantt Chart
- Average Waiting Time
- Turnaround Time
- Throughput

Use Case 2 : Comparing Algorithms

The user wants to understand how different algorithms behave with the same input.

The user enters the following processes to the simulator:

- P1: arrival 3, burst 4
- P2: arrival 6, burst 3
- P3: arrival 8, burst 2

The user selects the algorithms FCFS, SRTF, RR(Q=2) and runs the simulation.

The output the user receives is different Gantt charts, waiting/turnaround times, and throughput values.

The user can now easily compare and contrast the different algorithms and how they respond to the same processes.

Use Case 3 : Instructor Demonstrations

An instructor wants a way to show the class how Priority scheduling works.

The instructor enters the following processes to the simulator:

- P1: arrival 0, burst 6, priority 3
- P2: arrival 1, burst 4, priority 1
- P3: arrival 2, brust 5, priority 2

The instructor selects the Priority scheduling algorithm and runs the simulation.

The instructor receives the output of a gantt chart showing P2 | P3 | P1, along with the values of the waiting/turnaround time, and throughput.

The instructor can now show the class in real time, how priority scheduling works along with visualizing the pros and cons.

Project Links

GitHub Repository: https://github.com/J-Koral/cpu-scheduling-simulator

GitHub Pages (Website): https://j-koral.github.io/cpu-scheduling-simulator/

Project Management Board: https://github.com/users/J-Koral/projects/1

Project Documents:
https://github.com/J-Koral/cpu-scheduling-simulator/tree/master/src/main/resources/SoftwareDocumentation